# Reto Técnico Frontend iO – App de Productos y Beneficios

## Resumen

Construir desde cero una app móvil en React Native con un Dashboard de dos tabs: **Mis productos**, donde el usuario ve sus cuentas y tarjetas y puede consultar los datos sensibles en una vista segura, y **Beneficios**, donde ve en un mapa los beneficios de sus tarjetas con un carrusel sincronizado. Ambas tabs se apoyan en una única librería nativa propia (Kotlin o Swift) que expone el mapa y la vista segura. La prueba incluye pruebas unitarias obligatorias y se sustenta en una reunión técnica.

**Competencias evaluadas**

- Integración JS ↔ Nativo (bridge, componentes nativos, eventos, callbacks).
- SOLID, Clean Architecture y Vertical Slicing (BIAN) por capabilities "Cards / Account Management" y "Beneficios".
- React / React Native avanzado (estado, hooks, memo, listas).
- Diseño, accesibilidad y performance.
- Seguridad móvil (secure storage, anti-screenshot, timeouts, logs).
- Pruebas unitarias y testabilidad del código.
- Documentación técnica clara.

## Condiciones de la prueba

| Condición | Definición |
|---|---|
| Plazo | 4 días calendario desde el envío del enunciado. |
| Plataforma | Una plataforma obligatoria: Android con Kotlin o iOS con Swift. La segunda plataforma suma como extra. |
| Obligatorio | Partes 1 y 2, pruebas unitarias, README y video. |
| Suma puntos | Extras valorados y Parte 3. |

| Condición | Definición |
|---|---|
| SDK de mapas | Se permiten los SDK oficiales de cada plataforma: MapKit (iOS) y Google Maps SDK (Android). |
| UI / UX | Diseño libre, pero se espera una UI atractiva y bien cuidada: jerarquía visual clara, estados de carga, vacío y error, transiciones fluidas y una experiencia intuitiva. |
| Datos sensibles | PAN, CVV y titular completos viven solo en el módulo nativo y nunca pasan por la capa JS. |
| Documentación | Se valora una buena documentación: README claro con instalación paso a paso, diagrama de arquitectura, decisiones técnicas y trade-offs, APIs públicas y eventos con ejemplos de uso, cómo ejecutar las pruebas y verificación de seguridad. |

## Parte 1 – App RN: Dashboard con tabs

Una sola app con un Dashboard y navegación por tabs inferiores. Las dos tabs comparten header, tema visual, manejo de estado y capa de datos, y se construyen sobre la misma arquitectura.

**Tab Mis productos**

- Listado de cuentas y tarjetas (mock) con datos enmascarados: PAN `**** **** **** 1234` y fecha MM/AA.
- Botón "Ver datos sensibles" en cada tarjeta. Al hacer tap:
  1. RN solicita o renueva un `secureToken` (mock o generado localmente).
  2. RN invoca el módulo nativo para abrir la "Card Secure View", enviando `cardId` y `secureToken`.

**Tab Beneficios**

- Renderizar los beneficios como marcadores en `<CustomMap />`.
- Mostrar solo los beneficios que aplican a las tarjetas del usuario (campo `applicableBrands`), conectando ambas tabs.
- Al tocar un marcador, mostrar el beneficio destacado en un carrusel horizontal.

- Sincronizar mapa y carrusel en ambas direcciones: tocar un marcador mueve el carrusel, y deslizar el carrusel resalta el marcador.

Ambas tabs deben ser adaptativas y accesibles (a11y), y conservar su estado al cambiar de una a otra.

Mock de tarjetas ( `cards.json` ):

```
[
  {
    "cardId": "card_001",
    "alias": "Crédito Visa",
    "maskedPan": "**** **** **** 1234",
    "brand": "VISA",
    "holder": "JUAN PEREZ",
    "expiry": "12/28",
    "accountId": "acc_101"
  }
]
```

Mock de beneficios ( `benefits.json` ):

```
[
  {
    "id": 1,
    "title": "Beneficio 1",
    "description": "Descripción del beneficio",
    "image": "https://...",
    "promotion": "20% off",
    "address": "Dirección aquí",
    "latitude": 0.0,
    "longitude": 0.0,
    "applicableBrands": ["VISA"]
  }
]
```

## Parte 2 – Librería nativa (Kotlin / Swift)

Una sola librería nativa propia, con nombre y namespace claros, que expone los dos componentes que usa la app: el mapa de la tab Beneficios y la vista segura de la tab Mis productos.

## 2.1 Mapa `<CustomMap />`

Componente nativo que renderiza un mapa y permite agregar marcadores dinámicos.

| Prop / evento | Tipo | Descripción |
|---|---|---|
| `markers` | `Marker[]` | Lista de objetos con coordenadas y datos asociados. |
| `selectedMarkerId` | `string` | Marcador resaltado, para la sincronización desde el carrusel. |
| `onMarkerPress` | `(markerId) => void` | Evento disparado al presionar un marcador. |

## 2.2 Card Secure View

Vista segura a pantalla completa que valida el token antes de mostrar cualquier dato sensible.

**Comportamiento**

- Validar el token con reglas simples (por ejemplo, firma HMAC simulada y TTL).
- Token válido: mostrar PAN completo, CVV, fecha y titular desde un mock interno del módulo nativo; estos datos nunca deben pasar por la capa JS.
- Token inválido o expirado: mostrar error y cerrar, o pedir reenvío del token.

**Requisitos de seguridad mínimos**

- Bloqueo de screenshots ( `FLAG_SECURE` en Android / protección equivalente en iOS).
- Auto-ocultar en background (blur al pasar a background).
- Timeout de sesión (30–60 s) y botón "Ocultar".
- Sin logs de datos sensibles (verificación documentada en el README).

**Eventos expuestos a JS (sugeridos)**

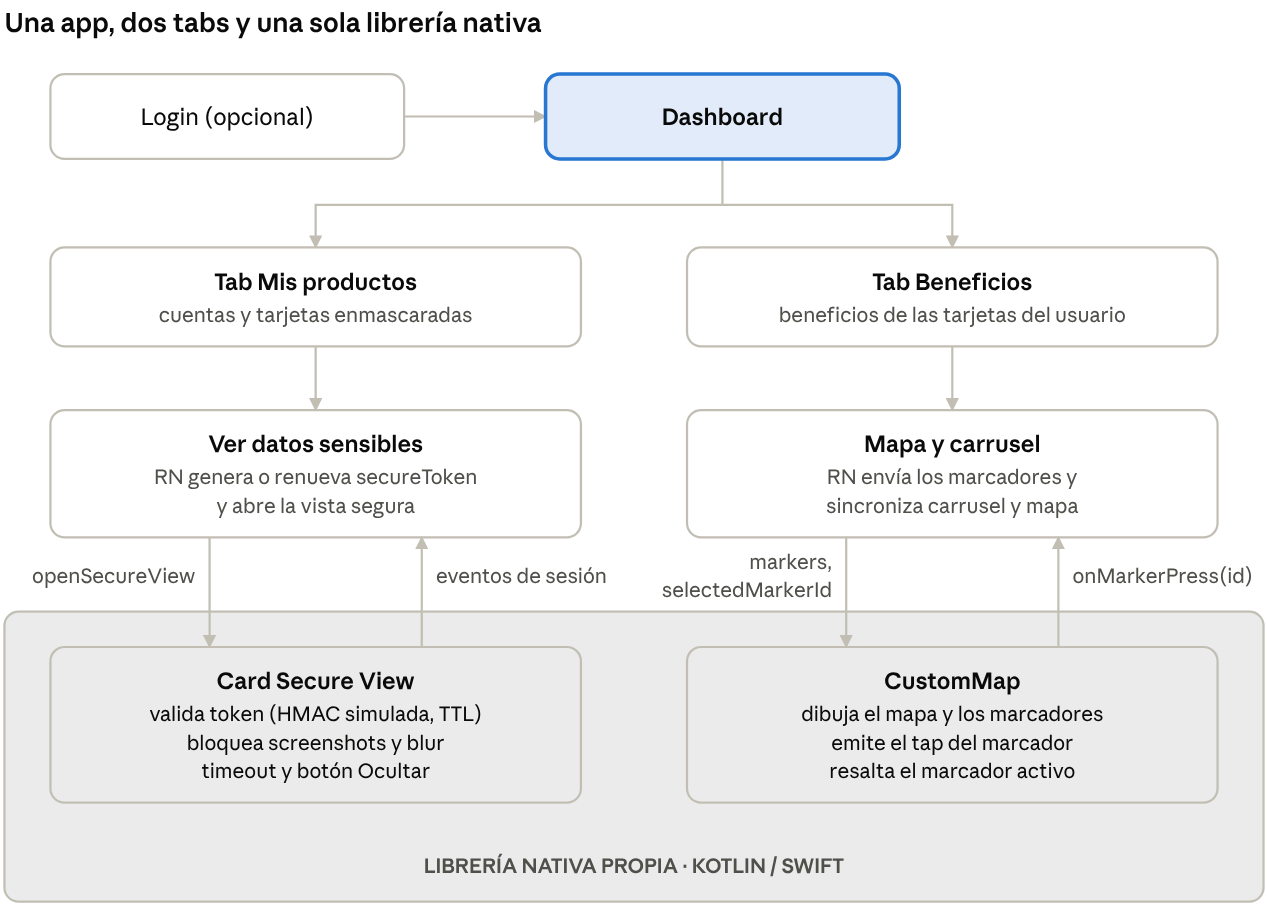
| Evento | Cuándo se emite |
|---|---|
| `onSecureViewOpened(cardId)` | Al abrir la vista segura. |
| `onValidationError(code, message)` | Token inválido o expirado ( `TOKEN_EXPIRED` , `TOKEN_INVALID` ). |

| Evento | Cuándo se emite |
|---|---|
| `onCardDataShown(cardId)` | Cuando los datos ya se renderizaron. |
| `onSecureViewClosed(cardId, reason)` | Al cerrar la vista ( `USER_DISMISS` , `TIMEOUT` ). |

### 2.3 Extras valorados (ambos módulos)

- Publicar la librería en npm o GitHub Packages.
- Usar nombres y namespaces claros para evitar conflictos.
- Que ambos módulos funcionen tanto en iOS como en Android.

## Flujo esperado




app iO · Dashboard con dos tabs sobre una librería nativa

Tras el login, el usuario llega al Dashboard y navega entre las dos tabs. En Mis productos, RN genera el token y abre la Card Secure View, que valida y emite los eventos de sesión. En Beneficios, RN envía los marcadores al `<CustomMap />`, recibe `onMarkerPress` y devuelve `selectedMarkerId`, para que mapa y carrusel queden sincronizados.

## Pruebas unitarias (obligatorias)

Las pruebas unitarias son obligatorias: se ejecutan con un solo comando (`npm test` o `yarn test`) y se documentan en el README.

| Capa | Qué se debe probar | Herramientas sugeridas |
|---|---|---|
| Tab Mis productos (RN) | Render de cuentas y tarjetas, enmascarado del PAN y fecha MM/AA, acción "Ver datos sensibles" | Jest + React Native Testing Library |
| Tab Beneficios (RN) | Transformación de `benefits.json` a marcadores, sincronización mapa ↔ carrusel, etiquetas de accesibilidad | Jest + React Native Testing Library |
| Token (RN) | Generación y renovación del `secureToken`, expiración | Jest |
| Bridge (RN) | `openSecureView`, `onMarkerPress` y demás eventos, con los módulos nativos mockeados | Jest |
| Validación de token (nativo) | Token válido, inválido, expirado, firma HMAC incorrecta | JUnit (Kotlin) / XCTest (Swift) |
| Mapa y sesión segura (nativo) | Mapeo de marcadores, emisión de eventos, timeout y motivo de cierre (deseable) | JUnit / XCTest |

**Requisitos mínimos**

- Casos de éxito y de error para cada evento (`TOKEN_EXPIRED`, `TOKEN_INVALID`, `USER_DISMISS`, `TIMEOUT`).
- Casos de borde del mapa: lista vacía de marcadores, coordenadas inválidas, marcador inexistente.
- Ninguna prueba ni snapshot debe contener ni imprimir datos sensibles reales.

- Reporte de cobertura generado ( `--coverage` ), con una cobertura mínima recomendada del 70% en la capa de React Native.
- Las pruebas deben pasar en limpio sobre la rama entregada.

## Parte 3 – Opcional para subir de nivel

- **Autenticación simple con Firebase:** login con usuario predefinido (incluir credenciales) y navegación al Dashboard tras un login correcto.
- **State management:** integrar una solución de manejo de estado.
- **Performance:** optimizar memoria, listas, carrusel y renders.

## Criterios de evaluación

| Área | Peso | Detalles |
|---|---|---|
| Ingeniería de software | 35% | Código limpio, SOLID, Clean Architecture, Vertical Slicing (BIAN) por capability, nombres consistentes, testabilidad y pruebas unitarias. |
| Funcionalidad | 25% | Cumplimiento exacto de requerimientos, sincronización mapa ↔ carrusel, flujos de validación y error, comunicación JS ↔ nativo sin errores. |
| Diseño y UX | 15% | Interfaz intuitiva, adaptativa y accesible (a11y), UI atractiva y cuidada con buena experiencia de usuario. |
| Documentación | 15% | README con instalación, uso, APIs públicas, eventos, decisiones técnicas y verificación de seguridad. |
| Performance | 10% | Uso eficiente de memoria y render, listas y carrusel optimizados, sin renders innecesarios. |

La sustentación es requisito para aprobar: si el candidato no puede explicar y defender su código, la evaluación no es válida, independientemente del puntaje obtenido.

## Restricciones

- No usar wrappers ni librerías de React Native que ya resuelvan mapas (p. ej. `react-native-maps`), secure card views o bridges equivalentes.
- Sí se permiten los SDK oficiales de cada plataforma (MapKit, Google Maps SDK) y las APIs del sistema.
- Los módulos nativos y su bridge deben ser propios.
- Entregar dentro del plazo definido.

## Entrega

Enviar por correo con el asunto **[Frontend] Entrega Reto iO - [Nombre Candidato]** incluyendo:

- [ ] Repositorio en GitHub con el código de la librería.
- [ ] Código de la app de ejemplo.
- [ ] Pruebas unitarias y reporte de cobertura.
- [ ] README con instalación, configuración, APIs públicas, eventos, cómo ejecutar las pruebas y verificación de seguridad.
- [ ] Video demostrando la app.
- [ ] Credenciales de prueba (solo si se implementa la Parte 3).

## Sustentación

Tras la entrega se agendará una reunión técnica en la que el candidato sustentará su solución ante el equipo evaluador. Agenda propuesta:

1. Demo en vivo de la app completa: navegación entre tabs, sincronización mapa ↔ carrusel y flujo de la vista segura (token válido, expirado, inválido, timeout y cierre manual).
2. Medidas de seguridad: bloqueo de screenshots, blur en background y ausencia de logs sensibles.
3. Arquitectura: capas, SOLID y Vertical Slicing por capability.
4. Diseño de los bridges y del contrato de eventos JS ↔ nativo.
5. Pruebas unitarias: estrategia, cobertura y ejecución en vivo.
6. Decisiones de accesibilidad y performance.

7. Preguntas del equipo evaluador.